

INTERNSHIP PROJECT REPORT

Serverless Personal To-Do List Application

Submitted By

Name : Bhavani M Ghorpade

June 2026

ABSTRACT

The Serverless Personal To-Do List Application is a cloud-native web application developed using Amazon Web Services (AWS) serverless technologies. The application is designed to manage daily tasks efficiently without maintaining physical or virtual servers. The project demonstrates implementation of scalable and cost-effective cloud infrastructure using AWS managed services.

The application uses AWS Lambda for backend processing, API Gateway for REST API communication, DynamoDB for storing To-Do items, and Amazon S3 for frontend website hosting. The frontend communicates with backend APIs using HTTP requests, while DynamoDB stores task information securely. The serverless architecture automatically manages scaling, monitoring, and infrastructure maintenance.

The application supports CRUD operations including task creation, retrieval, updating, and deletion. The project provides practical understanding of cloud-native application development, REST API integration, NoSQL databases, and frontend deployment using AWS cloud services.

OBJECTIVE

1. To develop a serverless web application using AWS cloud services.
2. To implement RESTful APIs using AWS API Gateway.
3. To create backend logic using AWS Lambda functions.
4. To store and retrieve task data using DynamoDB.
5. To host the frontend application using Amazon S3.
6. To implement CRUD operations for task management.

INTRODUCTION

Cloud computing has transformed modern software development by providing scalable and reliable infrastructure services through the internet. Traditional application deployment requires maintaining servers, configuring infrastructure, monitoring resources, and handling scalability manually. These operations increase operational complexity and infrastructure cost.

Serverless computing is a modern cloud computing model where developers focus only on writing application logic while cloud providers automatically manage infrastructure resources. This architecture eliminates server management overhead and improves scalability.

Amazon Web Services (AWS) provides several serverless services such as AWS Lambda, API Gateway, DynamoDB, and Amazon S3. These services allow developers to create scalable cloud-native applications efficiently and cost-effectively.

In this project, a Serverless Personal To-Do List Application is developed using AWS serverless technologies. The frontend application is hosted on Amazon S3, while API Gateway manages RESTful APIs. AWS Lambda processes backend business logic and DynamoDB stores user task data.

The application supports the following operations:

- Create Task
- Read Task
- Update Task
- Delete Task

The project demonstrates practical implementation of serverless architecture and modern cloud-native application deployment.

PROBLEM STATEMENT

Traditional task management applications generally require dedicated server infrastructure for backend processing, database hosting, scaling, monitoring, and deployment. Maintaining such infrastructure increases operational cost and requires continuous server administration.

Small-scale applications often suffer from unnecessary infrastructure overhead even when user traffic is low. Managing servers also introduces challenges related to security updates, scalability, and fault tolerance.

The objective of this project is to solve these issues using a serverless cloud-native architecture. By using AWS managed services such as Lambda, API Gateway, DynamoDB, and S3, the application eliminates the need for server management while maintaining scalability, reliability, and high availability.

The project aims to provide a lightweight, scalable, and cost-effective task management system capable of handling CRUD operations efficiently through serverless infrastructure.

LITERATURE SURVEY

Serverless computing has become one of the most important technologies in cloud computing because it reduces operational complexity and infrastructure maintenance. Research studies show that serverless applications improve scalability, reduce deployment cost, and simplify backend management.

AWS Lambda is commonly used for event-driven applications because it automatically scales according to workload demand. API Gateway simplifies API communication between frontend and backend services. DynamoDB provides highly scalable NoSQL storage with low latency and high availability.

Applications such as task management systems, chat systems, IoT applications, and notification services commonly use serverless architectures due to their flexibility and automatic scaling capabilities.

Organizations prefer serverless deployment because it provides:

- Automatic Scaling
- Reduced Infrastructure Management
- High Availability
- Pay-As-You-Use Billing
- Faster Application Deployment

The Serverless Personal To-Do List Application demonstrates practical implementation of these technologies using AWS managed services.

EXISTING SYSTEM

Existing task management systems are mostly developed using traditional monolithic architectures where frontend, backend, and database services are hosted on dedicated servers.

The limitations of traditional systems include:

- High infrastructure maintenance cost.
- Manual server scaling requirements.
- Complex deployment procedures.
- Increased downtime during maintenance.
- Limited scalability during traffic spikes.
- Continuous monitoring and server patching.

Traditional hosting models are not cost-efficient for small-scale applications because server resources remain active even during low usage periods.

Serverless architectures overcome these issues by providing automatic scaling, event-driven execution, and pay-per-use billing models.

PROPOSED SYSTEM

The proposed system uses AWS serverless technologies to create a scalable and reliable To-Do List Application.

The system consists of:

- Frontend hosted on Amazon S3.
- REST APIs managed using API Gateway.
- Backend logic executed using AWS Lambda.
- NoSQL database storage using DynamoDB.
- IAM-based access management for security.

The proposed architecture eliminates server management responsibilities and provides automatic scaling with improved reliability.

Major features of the proposed system include:

1. Serverless backend execution.
2. RESTful API integration.
3. Cloud-native NoSQL database.
4. Static frontend hosting.
5. Automatic infrastructure scaling.
6. Reduced deployment cost.

TOOLS AND TECHNOLOGIES USED

Tool / Technology	Purpose
AWS Lambda	Backend Logic Execution
API Gateway	REST API Communication
DynamoDB	NoSQL Database Storage
Amazon S3	Frontend Website Hosting
Python	Backend Programming
HTML	Frontend Structure
CSS	Frontend Styling
JavaScript	Frontend Functionality
AWS IAM	Access and Permission Management
REST API	Communication Protocol

AWS SERVICES DESCRIPTION

AWS Lambda

AWS Lambda is a serverless compute service that executes backend code without requiring server management. Developers upload application code while AWS automatically manages execution environments, scaling, monitoring, and resource allocation.

Advantages of AWS Lambda:

- Automatic scaling.
- Event-driven execution.
- No server maintenance.
- High availability.
- Cost-effective pricing.

Amazon API Gateway

API Gateway is a managed AWS service used to create, publish, monitor, and secure REST APIs. It acts as an interface between frontend applications and backend Lambda functions.

Features include:

- REST API support.
- Request routing.
- Authentication support.
- API monitoring.
- CORS configuration.

Amazon DynamoDB

DynamoDB is a fully managed NoSQL database service designed for low latency and high scalability. It stores application data in key-value format.

Advantages include:

- High performance.
- Automatic scaling.

- Fully managed infrastructure.
- Flexible schema design.
- High availability.

Amazon S3

Amazon Simple Storage Service (S3) is used for object storage and static website hosting.

Features include:

- Static website hosting.
- High durability.
- Public content delivery.
- Scalable object storage.
- Cost-efficient hosting.

SYSTEM ARCHITECTURE

The architecture of the application consists of frontend, backend, API communication, and database components.

1. User accesses frontend website hosted on Amazon S3.
2. Frontend sends HTTP requests to API Gateway.
3. API Gateway triggers AWS Lambda functions.
4. Lambda processes CRUD operations.
5. DynamoDB stores and retrieves task data.
6. API Gateway returns response back to frontend.

Advantages of the architecture include:

- High Scalability
- Reduced Infrastructure Management
- Automatic Resource Allocation

- Improved Reliability
- Cost-Efficient Deployment

The following figure shows the overall architecture of the serverless To-Do List Application.

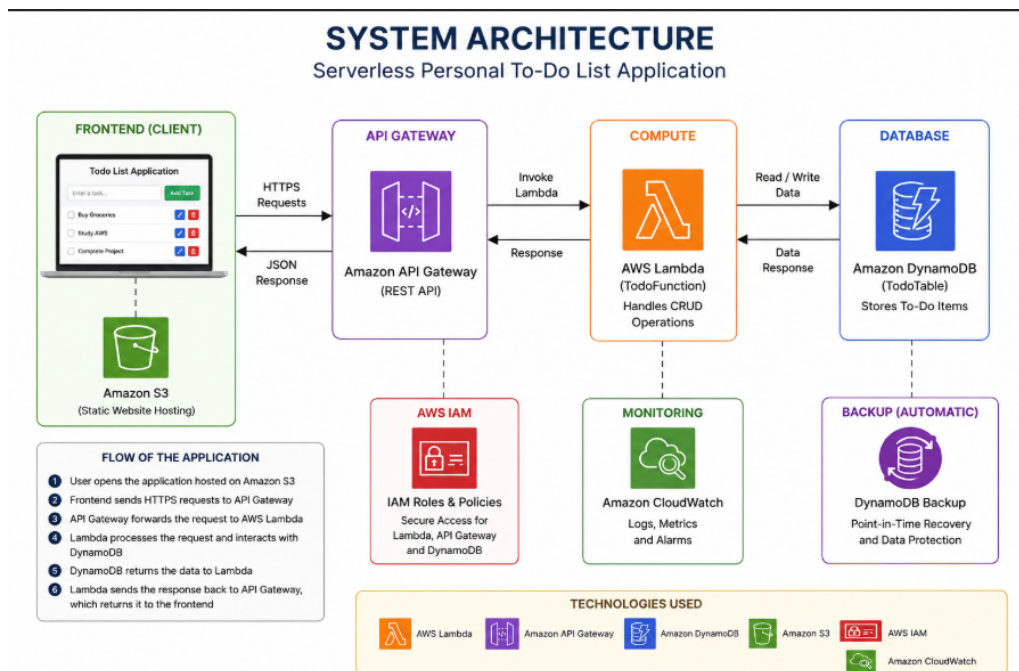


Figure 1: System Architecture

METHODOLOGY

Step 1: Create DynamoDB Table

A DynamoDB table named **ToDoTable** was created with partition key **id** of String datatype.

The table stores task information such as Task ID and Task Description. DynamoDB provides high scalability, low latency, and managed NoSQL storage.

The following figure shows the DynamoDB table configuration.

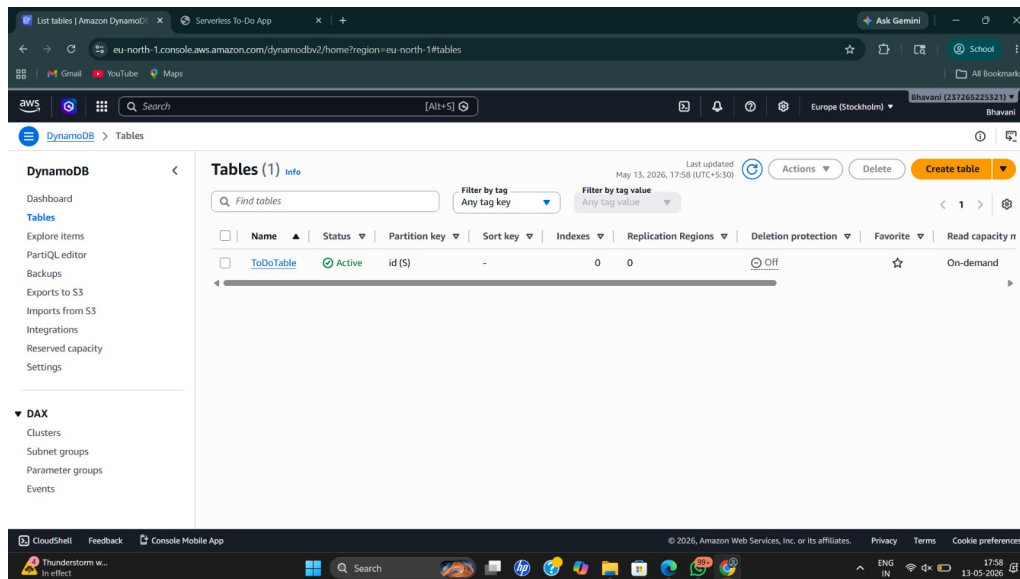


Figure 2: DynamoDB Table Configuration

Step 2: Create Lambda Function

A Lambda function named **ToDoFunction** was created using Python runtime.

The function handles GET, POST, PUT, and DELETE requests. Lambda automatically scales according to incoming requests and eliminates server maintenance.

The following figure shows the Lambda function created in AWS Console.

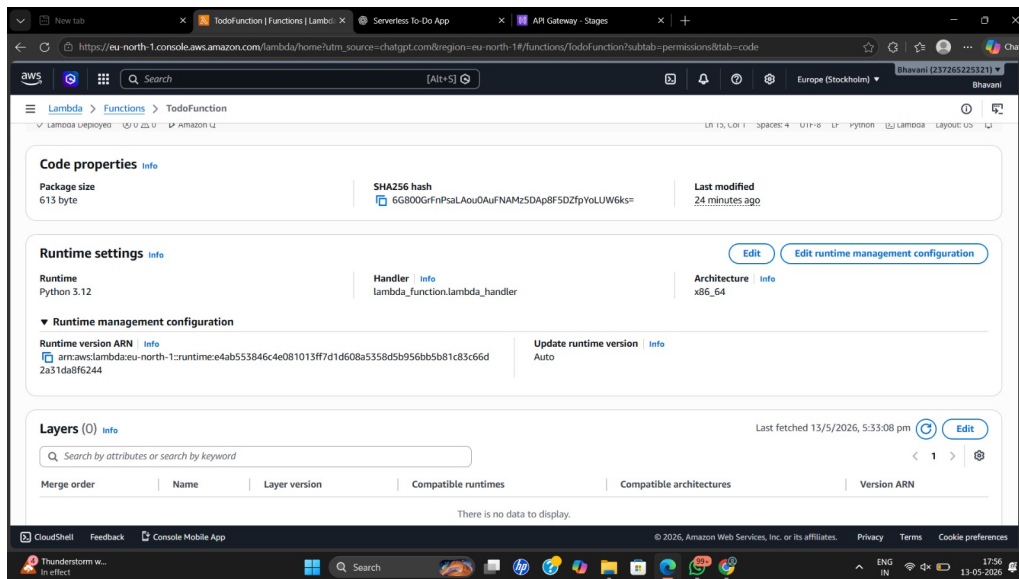


Figure 3: AWS Lambda Function

Step 3: Configure IAM Permissions

The **AmazonDynamoDBFullAccess** policy was attached to the Lambda execution role. This allowed Lambda functions to securely access DynamoDB resources.

IAM improves security and controlled access management for AWS services.

Step 4: Create API Gateway

A REST API named **TodoAPI** was created.

Configured methods include:

- GET
- POST
- PUT
- DELETE

API Gateway handled request routing and frontend-backend communication.

The following figure shows API Gateway configuration.

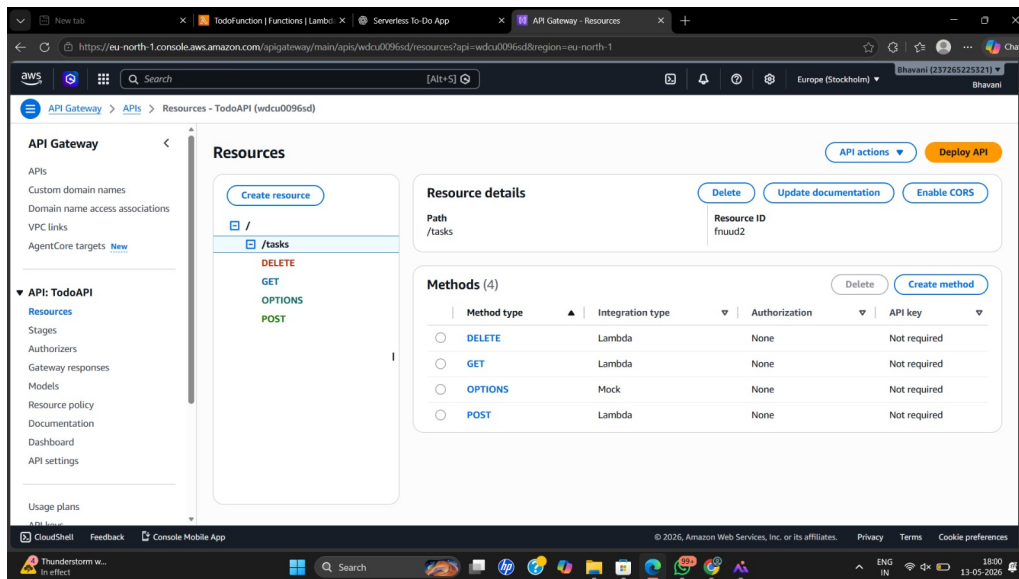


Figure 4: API Gateway Configuration

Step 5: Develop Frontend Application

Frontend application was developed using HTML, CSS, and JavaScript.

The frontend provides the following functionalities:

- Add Task
- View Tasks
- Update Tasks
- Delete Tasks

JavaScript Fetch API was used for backend communication.

Step 6: Host Frontend Using S3

Frontend files were uploaded to Amazon S3.

Static Website Hosting was enabled to make the application publicly accessible.

The following figure shows Amazon S3 static website hosting configuration.

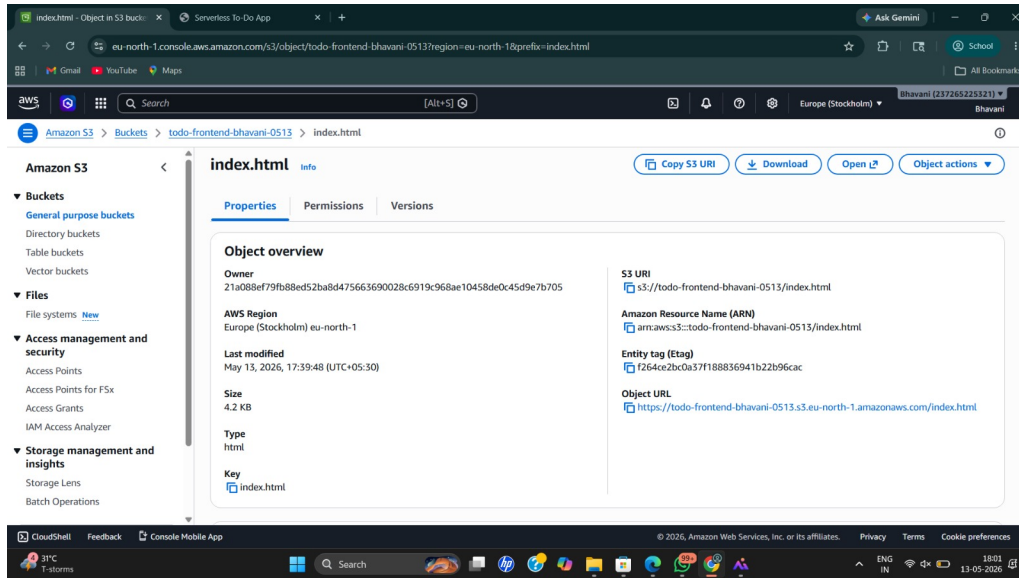


Figure 5: Amazon S3 Static Website Hosting

DATA FLOW OF APPLICATION

The workflow of the application follows a client-serverless architecture model.

1. User opens the frontend application hosted on Amazon S3.
2. User enters task information through the web interface.
3. JavaScript sends HTTP requests using Fetch API.
4. API Gateway receives the request.
5. API Gateway invokes the Lambda function.
6. Lambda validates request data.
7. DynamoDB performs required database operations.
8. Lambda generates JSON response.
9. API Gateway forwards response to frontend.
10. Frontend updates task list dynamically.

The complete workflow ensures smooth communication between frontend and backend services.

IMPLEMENTATION DETAILS

Frontend Implementation

The frontend application was designed using HTML, CSS, and JavaScript.

HTML was used to create the structure of the application interface. CSS improved visual appearance and layout styling. JavaScript handled dynamic interaction and communication with backend APIs.

Features implemented in frontend:

- Dynamic task rendering.
- API communication using Fetch API.
- Input handling.
- Task display updates.
- User interaction handling.

Backend Implementation

AWS Lambda was used for backend logic implementation.

The backend performs:

- Request processing.
- CRUD operation handling.
- Database communication.
- JSON response generation.
- Error handling.

Python programming language was used because of its simplicity and AWS SDK support.

Database Implementation

DynamoDB stores task information in NoSQL format.

Each task contains:

- Task ID
- Task Description

- Timestamp

The database provides low-latency access and automatic scalability.

SOURCE CODE

Lambda Function Code

```
1
2 import json
3 import boto3
4 import uuid
5
6 dynamodb = boto3.resource('dynamodb')
7 table = dynamodb.Table('TodoTable')
8
9 def lambda_handler(event, context):
10
11     method = event['httpMethod']
12
13     if method == 'GET':
14
15         response = table.scan()
16
17         return {
18             'statusCode': 200,
19             'headers': {
20                 'Access-Control-Allow-Origin': '*'
21             },
22             'body': json.dumps(response['Items'])
23         }
24
25     elif method == 'POST':
26
27         body = json.loads(event['body'])
28
29         item = {
30             'id': str(uuid.uuid4()),
31             'task': body['task']
32         }
```

```

33
34     table.put_item(Item=item)
35
36     return {
37         'statusCode': 200,
38         'headers': {
39             'Access-Control-Allow-Origin': '*'
40         },
41         'body': json.dumps({'message': 'Task Added'})
42     }

```

FRONTEND CODE

```

1
2 <!DOCTYPE html>
3
4 <html>
5
6 <head>
7
8 <title>Todo Application</title>
9
10 </head>
11
12 <body>
13
14 <h1>Todo List Application</h1>
15
16 <input type="text" id="taskInput">
17
18 <button onclick="addTask()">Add Task</button>
19
20 <ul id="taskList"></ul>
21
22 <script>
23
24 const apiUrl = "YOUR_API_GATEWAY_URL/tasks";
25
26 async function loadTasks() {

```

```

27
28     const response = await fetch(apiUrl);
29
30     const data = await response.json();
31
32     const list = document.getElementById("taskList");
33
34     list.innerHTML = "";
35
36     data.forEach(task => {
37
38         const li = document.createElement("li");
39
40         li.innerText = task.task;
41
42         list.appendChild(li);
43
44     });
45
46 }
47
48 async function addTask() {
49
50     const task = document.getElementById("taskInput").value;
51
52     await fetch(apiUrl, {
53
54         method: "POST",
55
56         headers: {
57             "Content-Type": "application/json"
58         },
59
60         body: JSON.stringify({task})
61
62     });
63
64     loadTasks();
65

```

```
66 | }
67 |
68 | loadTasks();
69 |
70 | </script>
71 |
72 | </body>
73 |
74 | </html>
```

TESTING AND VALIDATION

Different testing procedures were performed to ensure proper application functionality.

API Testing

API Gateway endpoints were tested using browser requests and frontend integration.

Verified operations include:

- GET request testing.
- POST request testing.
- PUT request testing.
- DELETE request testing.

Frontend Testing

Frontend validation included:

- User input testing.
- Button functionality testing.
- API response handling.
- Dynamic task rendering.

Database Testing

DynamoDB table operations were verified to ensure proper data storage and retrieval.

The database correctly handled:

- Item insertion.
- Item updates.
- Item deletion.
- Data retrieval.

Performance Testing

The serverless architecture demonstrated automatic scaling and fast response times under multiple requests.

RESULTS AND OUTPUT

The Serverless Personal To-Do List Application was successfully deployed using AWS cloud services.

Results achieved include:

- Successful deployment of Lambda backend.
- API Gateway exposed REST APIs correctly.
- DynamoDB stored task data successfully.
- Frontend hosted on Amazon S3 worked correctly.
- CRUD operations executed successfully.
- Automatic scaling achieved using serverless architecture.

The following figure shows the final application output.

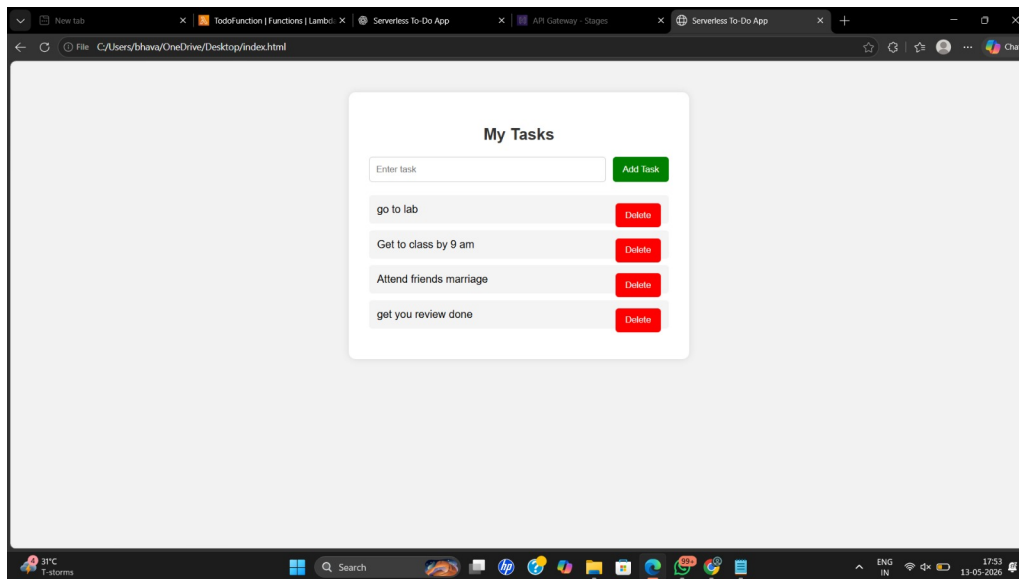


Figure 6: Application Output

ADVANTAGES OF SERVERLESS ARCHITECTURE

1. No server maintenance required.
2. Automatic scaling based on workload.
3. Reduced operational cost.
4. High availability and reliability.
5. Faster deployment process.
6. Improved resource utilization.
7. Easy integration with AWS services.
8. Better scalability and performance.

CHALLENGES FACED

Several challenges were encountered during implementation of the project.

- Configuring API Gateway methods properly.
- Managing IAM permissions correctly.
- Enabling CORS for frontend communication.
- Integrating Lambda with DynamoDB.

- Configuring S3 public access policies.
- Debugging REST API responses.

These issues were resolved using AWS documentation and testing procedures.

PERFORMANCE ANALYSIS

The serverless architecture provided efficient performance with automatic resource scaling.

Observed benefits include:

- Reduced infrastructure overhead.
- Fast API response times.
- Automatic scalability.
- Improved resource utilization.
- Reduced operational cost.

Lambda functions executed efficiently without requiring dedicated server resources. DynamoDB handled database operations with low latency and high availability.

The application architecture is capable of supporting multiple concurrent users while maintaining reliable performance.

FUTURE ENHANCEMENTS

Future improvements can include:

- Add AWS Cognito authentication.
- Improve frontend using ReactJS.
- Add task priorities and categories.
- Implement notifications and reminders.
- Use AWS SAM for deployment automation.
- Add mobile-responsive design.
- Implement task search functionality.

CONCLUSION

The Serverless Personal To-Do List Application was successfully developed and deployed using AWS cloud services. The project demonstrated practical implementation of serverless architecture using AWS Lambda, API Gateway, DynamoDB, and Amazon S3.

The application successfully performed CRUD operations without maintaining servers. AWS services automatically handled infrastructure management, scaling, and monitoring, making the application scalable, reliable, and cost-efficient.

The project provided practical understanding of cloud-native application deployment, REST API integration, NoSQL databases, frontend hosting, and serverless computing concepts. The final application achieved reduced operational complexity and efficient cloud deployment using AWS managed services.

REFERENCES

1. AWS Official Documentation
2. AWS Lambda Developer Guide
3. Amazon DynamoDB Documentation
4. Amazon API Gateway Documentation
5. Amazon S3 Documentation
6. Python Official Documentation
7. REST API Design Guidelines
8. Cloud Computing Research Papers